

Module - 01Concept

Conditional loops in C programming are control structures that repeatedly execute a block of code as long as a specified test condition evaluates to true. They prevent you from writing duplicate code lines and automatically terminate once the target condition is met or becomes false.

In C, any non-zero value represent true, while zero represent false. C provides three distinct types of loops grouped into two execution categories :-

- The While loop (Entry - Controlled) —

The while loop keeps executing a block of statements as long as its condition remains true. Use this when you do not know the exact no. of iterations beforehand.

Syntax :-

```
while (condition) {  
    // loop body  
}
```

Ex:-

```
int i = 1;  
while (i <= 3)  
    printf("%d", i);  
    i++;  
}
```

output : 1 2 3

• The for loop (Entry Controlled) —

The for loop bundles initialization, condition checking and variable updates into a single line. Use this when the total no. of iterations is known ahead of time.

Syntax :-

```
for (Initialization ; condition ; updation )  
    // loop body  
{
```

Example :-

```
for (int i=1 ; i<=3 ; i++) {  
    Print ("x", i);  
}
```

// output :- 1 2 3

• The do-while loop (Exit Controlled) —

The do-while loop executes its body code block completely before evaluating the truth condition. Use this when the operation must execute at least once.

Syntax :-

```
do {
```

```
    // loop body
```

```
} while (condition);
```


Example :-

```
int i = 5;
```

```
do {
```

```
    printf("%d", i);
```

```
    i++;
```

```
} while (i <= 3);
```

Output : 5

Patterns Solution -

1. Square

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n=5;
```

```
    for (int i=0; i<n; i++) {
```

```
        for (int j=0; j<n; j++) {
```

```
            printf("* ");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

main.c		Run	Output	Clear
1	#include <stdio.h>		*****	
2			*****	
3	int main() {		*****	
4	int n = 5;		*****	
5	for (int i = 0; i < n; i++) {		*****	
6	for (int j = 0; j < n; j++) {			
7	printf("*");			
8	}		=== Code Execution Successful ===	
9	printf("\n");			
10	}			
11	return 0;			
12	}			
13				

2. Hollow Square

```
* * * * *
*           *
*           *
*           *
* * * * *
```

```
#include <stdio.h>
int main() {
    int n=5;
    for (int i=0; i<n; i++) {
        for (int j=0; j<n; j++) {
            if (i==0 || i==n-1 || j==0 || j==n-1) {
                printf("*");
            } else {
                printf(" ");
            }
        }
        printf("\n");
    }
    return 0;
}
```

main.c



Run

Output

Cle

```
1  #include <stdio.h>
2
3  int main() {
4      int n = 5;
5      for (int i = 0; i < n; i++) {
6          for (int j = 0; j < n; j++) {
7              // Print stars for borders, spaces
              // inside
8              if (i == 0 || i == n - 1 || j == 0
                  || j == n - 1) {
9                  printf("*");
10             } else {
11                 printf(" ");
12             }
13         }
14         printf("\n");
15     }
16     return 0;
17 }
```

```
*****
*  *
*  *
*  *
*****
```

=== Code Execution Successful ===

3

Triangle

```

*
* *
* * *
* * * *
* * * * *

```

```
#include <stdio.h>
```

```
int main()
```

```
int n=5;
```

```
for (int i=1; i<=n; i++) {
```

```
for (int j=1; j<=i; j++) {
```

```
    printf("*");
}
```

```
    printf printf("\n");
}
```

```
return 0;
```

```
}
```


main.c



Run

Output

Clear

```
1  #include <stdio.h>
2
3  int main() {
4      int n = 5;
5      for (int i = 1; i <= n; i++) {
6          for (int j = 1; j <= i; j++) {
7              printf("*");
8          }
9          printf("\n");
10     }
11     return 0;
12 }
13
```

```
*
**
***
****
*****
```

=== Code Execution Successful ===

4. Inverted Triangle

```
* * * * *
* * * *
* * *
* *
*
```

```
#include <stdio.h>
int main() {
    int n = 5;
    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            printf(" * ");
        }
        printf("\n");
    }
    return 0;
}
```

main.c



Run

Output

Clear

```
1 #include <stdio.h>
2
3 int main() {
4     int n = 5;
5     for (int i = n; i >= 1; i--) {
6         for (int j = 1; j <= i; j++) {
7             printf("*");
8         }
9         printf("\n");
10    }
11    return 0;
12 }
13
```

```
*****
****
***
**
*
```

=== Code Execution Successful ===

5. Pyramid

```

      *
     * *
    * * *
   * * * *
  * * * * *
 * * * * *

```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n=5;
```

```
    for (int i=1; i<=n; i++){
```

```
        for (int j=1; j<=n-i; j++){
```

```
            printf(" ");
```

```
        }
```

```
        for (int k=1; k<=(2*i-1); k++){
```

```
            printf("*");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

main.c



Run

Output

Clear

```
1 #include <stdio.h>
2
3 int main() {
4     int n = 5;
5     for (int i = 1; i <= n; i++) {
6         // Print leading spaces
7         for (int j = 1; j <= n - i; j++) {
8             printf(" ");
9         }
10        // Print stars
11        for (int k = 1; k <= (2 * i - 1); k++) {
12            printf("*");
13        }
14        printf("\n");
15    }
16    return 0;
17 }
```

```
*
***
*****
*****
*****
```

=== Code Execution Successful ===

6. Inverted Pyramid

```

* * * * *
  * * * *
    * * *
      * *
        *

```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n=5;
```

```
    for (int i=n; i>=1; i--) {
```

```
        for (int j=1; j<=n-i; j++) {
```

```
            printf(" ");
```

```
        }
```

```
        for (int k=1; k<=(2*i-1); k++) {
```

```
            printf("* ");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

main.c	   Run	Output Clear
<pre>1 #include <stdio.h> 2 3 int main() { 4 int n = 5; 5 for (int i = n; i >= 1; i--) { 6 // Print leading spaces 7 for (int j = 1; j <= n - i; j++) { 8 printf(" "); 9 } 10 // Print stars 11 for (int k = 1; k <= (2 * i - 1); k++) { 12 printf("*"); 13 } 14 printf("\n"); 15 } 16 return 0; 17 } 18</pre>	<pre>***** ***** ***** *** * === Code Execution Successful ===</pre>	

7. Half Diamond

```
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
```

```
#include <stdio.h>
int main() {
    int n=5;
    for (int i=1; i<=n; i++) {
        for (int j=1; j<=i; j++) {
            printf("*");
        }
        printf("\n");
    }
    for (int i=n-1; i>=1; i--) {
        for (int j=1; j<=i; j++) {
            printf("*");
        }
        printf("\n");
    }
    return 0;
}
```

main.c



Run

Output

Clear

```
1 #include <stdio.h>
2
3 int main() {
4     int n = 5;
5     // Upper half
6     for (int i = 1; i <= n; i++) {
7         for (int j = 1; j <= i; j++) {
8             printf("*");
9         }
10        printf("\n");
11    }
12    // Lower half
13    for (int i = n - 1; i >= 1; i--) {
14        for (int j = 1; j <= i; j++) {
15            printf("*");
16        }
17        printf("\n");
18    }
19    return 0;
20 }
```

```
*
**
***
****
*****
****
***
**
*
```

=== Code Execution Successful ===

```

      *
    * * *
  * * * * *
* * * * * * *
* * * * * * *
  * * * * *
    * * *
      *

```

```
#include <stdio.h>
```

```
int main() {
```

```
    int n=5;
```

```
    for (int i=1; i<=n; i++) {
```

```
        for (int j=1; j<=n-i; j++) {
```

```
            printf(" ");
```

```
        }
```

```
        for (int k=1; k<=(2*i-1); k++) {
```

```
            printf("*");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    for (int i=n-1; i>=1; i--) {
```

```
        for (int j=1; j<=n-i; j++) {
```

```
            printf(" ");
```

```
        }
```

```
        for (int k=1; k<=(2*i-1); k++) {
```

```
            printf("*");
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```

main.c



Run

Output

Clear

```
1 #include <stdio.h>
2
3 int main() {
4     int n = 5;
5     // Upper half (including center row)
6     for (int i = 1; i <= n; i++) {
7         for (int j = 1; j <= n - i; j++) {
8             printf(" ");
9         }
10        for (int k = 1; k <= (2 * i - 1); k++) {
11            printf("*");
12        }
13        printf("\n");
14    }
15    // Lower half
16    for (int i = n - 1; i >= 1; i--) {
17        for (int j = 1; j <= n - i; j++) {
18            printf(" ");
19        }
20        for (int k = 1; k <= (2 * i - 1); k++) {
21            printf("*");
22        }
23        printf("\n");
24    }
25    return 0;
26 }
```

```
 *
 ***
 *****
 *******
 *****
 *****
 *****
 ***
 *
```

=== Code Execution Successful ===